Introduction

This project applies frequent-itemset mining techniques to the domain of cinema: each movie is treated as a *basket*, and its four lead actors as the *items* it contains. The goal is to discover groups of actors who recur together across multiple films – effectively, recurring casts – by means of the **Apriori** and **PCY** (Park-Chen-Yu) algorithms, complemented by **association rule** mining.

The structure of this report is as follows: Section 1 describes the dataset and the parts of it used; Section 2 explains how the data were organized and pre-processed; Section 3 presents the algorithms and their implementation; Section 4 discusses how the solution scales with data size; Section 5 describes the experiments performed; and Section 6 reports and discusses the results.

1 Dataset

The dataset used is the *IMDb Movies Dataset*, a Kaggle-hosted collection of the top 1000 movies ranked by IMDb rating. The file contains 16 columns describing each movie (title, year, genre, IMDb rating, director, cast, gross revenue, etc.).

Of these, only five columns were used for this project:

- `Series_Title`: used only for human-readable labelling of results, not for the analysis itself.
- `Star1`, `Star2`, `Star3`, `Star4`: the four lead actors of each movie, which constitute the *items* of the market-basket model.

All other columns (genre, rating, director, runtime, gross revenue, etc.) were not used, as they are not relevant to the market-basket formulation.

The dataset contains 1000 movies and, after extracting the four star columns, 2,709 distinct actors. No missing values were found in the four star columns (every movie has exactly four actors listed).

2 Data Organization and Pre-processing

Following the market-basket model, the data were organized as a list of *baskets*, one per movie. Each basket is represented as a Python `set` containing the names of the (up to) four lead actors of that movie.

The following pre-processing steps were applied before running the mining algorithms:

1. **Missing-value filtering.** For each movie, only the non-null values among `Star1`–`Star4` were kept, so that a movie with fewer than four credited stars would not contribute spurious empty items. In practice this dataset has no missing star values, but the code handles the general case.
2. **De-duplication within a basket.** Constructing each basket as a `set` rather than a list automatically removes the case of the same actor appearing twice in

the four star columns of one movie – four movies in the dataset exhibit this. It also removes any dependency on column order.

3. **No text normalization was required.** Actor names are already consistently capitalized and formatted; no case-folding or fuzzy-matching was necessary. In a noisier dataset this would be an additional step (e.g., resolving "Robert DeNiro" vs. "Robert De Niro").

No rows were dropped; all 1000 movies are included as baskets. The resulting in-memory representation is a list of 1000 sets, occupying well under 1 MB – small enough to be held fully in main memory.

3 Algorithms and Implementation

Two complementary frequent-itemset algorithms from the course were implemented and compared, plus an association-rule generation step.

3.1 Apriori

Apriori exploits the *monotonicity* (downward-closure) property: if an itemset is not frequent, no superset of it can be frequent. The algorithm proceeds level-by-level in itemset size k:

1. $k = 1$: count the support of every single item; keep those with support $\geq$ the minimum support threshold s (set L_1).
2. $k > 1$: generate candidates of size k by joining pairs of frequent itemsets of size $k-1$ that differ in exactly one item (*join step*); discard any candidate having a $(k-1)$-subset that is not itself frequent (*prune step*, the practical consequence of monotonicity); count the exact support of the surviving candidates over all baskets and keep those $\geq s$.
3. Repeat until no new frequent itemsets are found.

A relevant implementation detail concerns the counting step. Rather than testing every candidate against every basket (which would cost $O(|\text{candidates}| \times |\text{baskets}|)$), each basket is used to directly enumerate its own k-subsets via `itertools.combinations`, each of which is looked up in the candidate set in $O(1)$ via a hash set.

3.2 PCY (Park-Chen-Yu)

PCY optimizes the most expensive step of Apriori – finding frequent pairs – by reducing the size of the candidate set that must be tracked in the second pass, using two passes:

- **Pass 1**: in addition to counting single items (as in Apriori), every pair of items co-occurring in a basket is hashed into one of B buckets, and a per-bucket counter is incremented. A bucket is *frequent* if its total count is $\geq s$.
- **Pass 2**: a pair $\{i, j\}$ becomes a candidate only if *both* (a) i and j are individually frequent items, *and* (b) $\{i, j\}$ hashes to a frequent bucket. Only candidate pairs are exactly counted.

No false negatives. A genuinely frequent pair contributes, by itself, at least s increments to its own bucket (one per occurrence), so its bucket is guaranteed to be marked frequent; PCY can only discard pairs that are *not* frequent (it reduces false positives among the candidates), never a truly frequent pair. This guarantee was also verified empirically (Section 6).

3.3 Association Rules

An association rule has the form $I \to j$, where I is a set of items and j is a *single* item. For every frequent itemset of size ≥ 2 and every item j it contains, the rule $(I - \{j\}) \to j$ is generated, and two measures are computed:

$$\text{confidence}(I \to j) = \frac{\text{support}(I \cup \{j\})}{\text{support}(I)} \tag{1}$$

$$\text{interest}(I \to j) = \text{confidence}(I \to j) - \frac{\text{support}(\{j\})}{N} \tag{2}$$

where N is the total number of baskets. Interest measures how much more (or less) likely j is to occur given I, compared to its baseline frequency; a strongly negative interest would indicate that I *discourages* the presence of j.

3.4 Correctness Verification

As a sanity check on the implementation, the set of frequent pairs returned by Apriori and by PCY was compared directly for equality after every experimental run. The two algorithms produced *identical* sets of frequent pairs in all configurations tested (varying both the support threshold and the number of PCY buckets), which is the expected behaviour given PCY's no-false-negative guarantee, and gives confidence in the correctness of both implementations.

4 Scalability

Although the IMDb Top 1000 dataset fits comfortably in main memory, the design choices made in this project were guided by how the solution would behave on substantially larger data, consistent with the course's framing of cost in terms of *number of passes over the data* rather than raw CPU time.

Number of passes. Both Apriori and PCY require one pass per itemset size k: this project performs 3 passes (for $k = 1, 2, 3$) since no frequent 4-itemsets exist at the chosen support threshold. This is the dominant cost metric when the basket file must be streamed from disk: wall-clock cost grows roughly linearly with the number of passes times the size of the dataset.

Memory: the real bottleneck. The limiting resource for both algorithms is the size of the *candidate-counting table* that must reside in main memory during a pass.

For Apriori's pair-counting pass, this table has up to $O(|L_1|^2)$ entries. In this project $|L_1| = 271$, giving 36,585 candidate pairs – trivial for any modern machine. However, on a larger dataset (e.g., the full IMDb catalogue with tens of thousands of distinct actors), $|L_1|$ could grow by an order of magnitude, and the candidate table would grow *quadratically*, potentially exceeding main memory. This is precisely the scenario PCY is designed for: by discarding pairs whose hash bucket did not reach the support threshold, PCY keeps the second-pass candidate table strictly smaller – in our experiments, 86.7% smaller with 5000 buckets (Section 6).

Tuning the bucket count for scale. The number of hash buckets B is a direct, tunable trade-off between memory and filtering effectiveness. Too few buckets causes hash collisions to make most buckets "frequent", degenerating PCY into plain Apriori (observed in the bucket-count sweep, Section 6); a well-chosen B approaching the true number of frequent pairs yields the largest reduction. At larger scale, B should be chosen relative to the available memory budget and the expected number of pairs.

5 Description of the Experiments

Four experiments were run to characterize both the substantive findings and the behaviour of the algorithms:

1. **Main run** ($s = 3$): Apriori and PCY were run with a minimum support threshold of $s = 3$ movies. This threshold was chosen empirically: given that the most frequent single actor appears in only 17 movies, a percentage-based threshold (e.g., 1%) would yield no frequent itemsets at all; $s = 3$ is the lowest integer value that produces a non-trivial, interpretable set of results.
2. **Support-threshold sweep** ($s \in \{2, 3, 4, 5, 7, 10\}$): Apriori was re-run at each threshold to quantify how the number of frequent itemsets and total running time vary with s.
3. **PCY bucket-count sweep** ($B \in \{50, 200, 1000, 5000, 20000, 50000\}$, at $s = 3$): PCY's pair-finding step was re-run with each bucket count to measure the candidate-set-size reduction as a function of B, and to verify that PCY's output is identical to Apriori's regardless of B.
4. **Association rules** (confidence ≥ 0.5, at $s = 3$): all rules $I \to j$ satisfying the confidence threshold were generated and ranked by absolute interest.

6 Results and Discussion

6.1 Frequent itemsets

At $s = 3$, Apriori (and, identically, PCY) found 271 frequent single actors, 25 frequent pairs, and 3 frequent triples; no frequent quadruple exists at this threshold (Figure 1). This matches the *"tyranny of counting pairs"* pattern described in the course: each level shrinks sharply relative to the previous one, since every $(k{+}1)$-itemset requires all of its k-subsets to already be frequent.

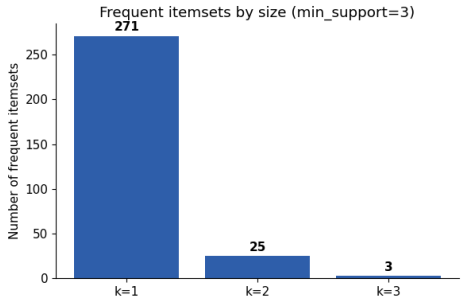

Figure 1: Number of frequent itemsets found per itemset size k, at $s = 3$.

The three frequent triples found are shown in Table 1; all three correspond to the recurring lead trio of a well-known film franchise.

Actor group	**Support**
Daniel Radcliffe, Emma Watson, Rupert Grint	5
Elijah Wood, Ian McKellen, Orlando Bloom	3
Carrie Fisher, Harrison Ford, Mark Hamill	3

Table 1: The three frequent triples found at $s = 3$ (*Harry Potter*, *The Lord of the Rings*, and the original *Star Wars* trilogy, respectively).

The ten most frequent actor pairs (Figure 2) are likewise dominated by recurring franchise casts and by long-running director-actor or co-star collaborations (e.g., Robert De Niro appearing repeatedly with several different co-stars across Scorsese films).

6.2 Sensitivity to the support threshold

Figure 3 shows both the number of frequent itemsets and the total Apriori running time as functions of s. Lowering s from 3 to 2 alone increases the number of frequent singletons from 271 to 641, and frequent pairs from 25 to 121; the resulting join step must consider on the order of $\binom{641}{2} \approx 205{,}000$ candidate pairs, which is reflected in the measured running time jumping from about 0.05 s ($s = 3$) to about 0.72 s ($s = 2$) – an over $10\times$ increase from a single unit decrease in threshold. This is a direct, measured illustration of the combinatorial explosion that motivates candidate-set-reduction techniques such as PCY, and supports the scalability discussion of Section 4: at lower thresholds (or with a larger item universe), the naïve candidate table can grow far faster than the dataset itself.

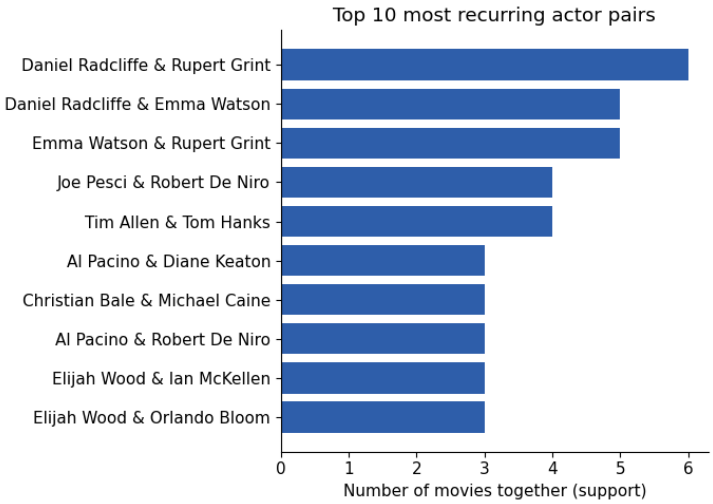

Figure 2: The 10 actor pairs with the highest support.

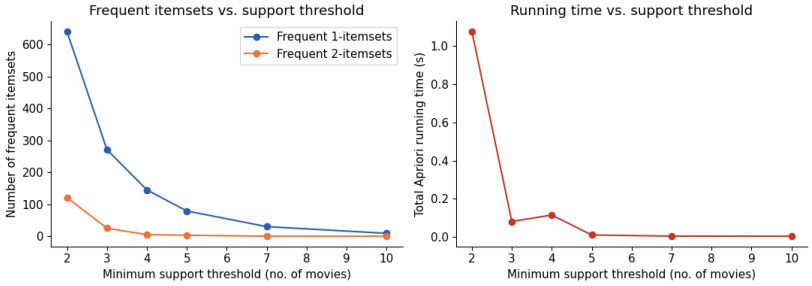

Figure 3: Number of frequent itemsets (left) and total Apriori running time (right) as a function of the minimum support threshold s.

6.3 Apriori vs. PCY: candidate-set reduction

With $B = 5000$ buckets, PCY's second-pass candidate set contains 4,854 pairs, versus 36,585 for plain Apriori – an 86.7% reduction (Figure 4) – while both algorithms return the exact same 25 frequent pairs, confirming the no-false-negative property empirically. On a dataset this small the absolute time saved is modest (PCY's two passes together take about 0.010 s versus Apriori's 0.13 s second pass alone), but the candidate-table-size reduction is the metric that matters for scalability, since it is what determines whether the second pass fits in memory at all on much larger data (Section 4).

The bucket-count sweep (Figure 5) shows this reduction is highly sensitive to B: with $B \leq 200$ buckets, PCY's candidate set is identical in size to Apriori's (the hash table is so coarse that essentially every bucket is frequent, so the filter has no effect), while increasing B to 20,000 reduces the candidate set to just 270 pairs – close to the true number of frequent pairs (25).

6.4 Association rules

20 rules $I \to j$ with confidence ≥ 0.5 were found; the ten with the highest absolute interest are shown in Table 2. All of them have confidence 1.00 and very high positive interest (close to +1), meaning that whenever the antecedent cast members appear

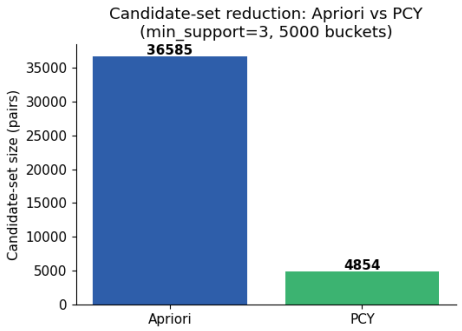

Figure 4: Candidate-set size for pair-finding: Apriori (all pairs of frequent items) vs. PCY (pairs surviving the hash-bucket filter), at $s = 3$, $B = 5000$.

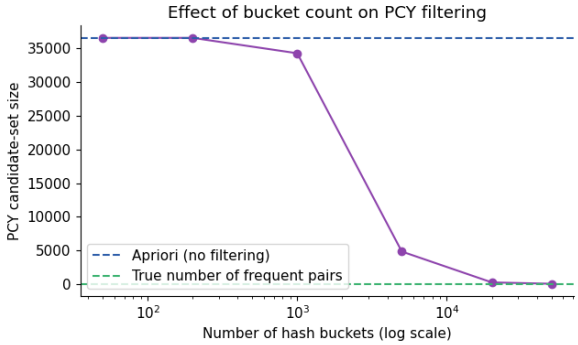

Figure 5: PCY candidate-set size as a function of the number of hash buckets B (log scale), at $s = 3$. The dashed lines mark Apriori's (unfiltered) candidate count and the true number of frequent pairs.

together, the consequent actor is essentially guaranteed to appear as well – again consistent with these being franchise/saga casts (Harry Potter, The Lord of the Rings, Star Wars) rather than coincidental co-occurrences. No rule with strongly negative interest was found among those passing the confidence threshold, which is expected: a negative-interest rule ("I discourages j") would by construction tend to have *low* confidence, and is therefore unlikely to also clear a 0.5 confidence bar.

6.5 Summary of findings

- The implementation correctly reproduces the expected theoretical behaviour of Apriori and PCY (identical output, no false negatives, monotonic shrinkage across itemset sizes).
- PCY achieves a substantial (up to 86.7%, and higher with more buckets) reduction in the size of the candidate table that must be tracked in the pair-counting pass, with the reduction directly tunable via the bucket count – the property that matters most for scaling to larger item universes.
- The combinatorial explosion of candidates as the support threshold is lowered was measured directly (over 10× slow-down for a one-unit decrease in s), reinforcing why candidate-reduction techniques are necessary in practice, not just in theory.

Antecedent (I)	**Consequent (j)**	**Support**	**Confidence**	**Interest**
Ian McKellen, Orlando Bloom	Elijah Wood	3	1.00	+0.997
Carrie Fisher, Harrison Ford	Mark Hamill	3	1.00	+0.997
Elijah Wood	Orlando Bloom	3	1.00	+0.996
Mark Hamill	Carrie Fisher	3	1.00	+0.996
Elijah Wood, Ian McKellen	Orlando Bloom	3	1.00	+0.996
Harrison Ford, Mark Hamill	Carrie Fisher	3	1.00	+0.996
Daniel Radcliffe	Rupert Grint	6	1.00	+0.994
Rupert Grint	Daniel Radcliffe	6	1.00	+0.994
John Turturro	Ethan Coen	3	1.00	+0.994
Daniel Radcliffe, Emma Watson	Rupert Grint	5	1.00	+0.994

Table 2: Top-10 association rules by absolute interest ($s = 3$, confidence ≥ 0.5).

- The substantive result – recurring franchise casts surfacing as the strongest frequent itemsets and association rules – is an intuitive and easily-verifiable sanity check that the pipeline (from raw CSV to frequent itemsets) is correct end-to-end.

Limitations. The IMDb Top 1000 dataset is itself a curated, high-quality subset of all films, and is small relative to what would stress-test the scalability properties discussed in Section 4. A larger, unfiltered movie database would likely produce a substantially larger item universe and a correspondingly larger gap between Apriori's and PCY's resource requirements, which this project's results can only extrapolate towards rather than directly measure.

7 Conclusion

This project implemented, from scratch, the Apriori and PCY algorithms for frequent-itemset mining, and applied them to discover recurring actor casts in the IMDb Top 1000 dataset. The implementation was verified for correctness against the course's theoretical guarantees (monotonicity, PCY's no-false-negative property), its scalability properties were analyzed both theoretically and through targeted experiments (support-threshold and bucket-count sweeps), and the resulting frequent itemsets and association rules were shown to correspond to recognizable, real-world recurring film casts.

References

[1] Anand Rajaraman, Jure Leskovec, and Jeffrey D. Ullman. *Mining of Massive Datasets*, Ch. 6. Cambridge University Press, 2014.